# Java on the Command Line

## Junior Level: Modules, Custom Runtimes and Installers

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-13

# Outline

1. The Module System
2. Modular JARs
3. Custom Runtimes: jlink
4. Installers: jpackage
5. Multi-Release JARs
6. Tuning the JVM from the Command Line
7. Build Tools and Modules
8. Summary

# The Module System

# Problems of the class path

The class path is a **flat list** of folders and JARs:

- **No hiding**: every `public` class is usable by everyone, even "internal" ones.
- **Missing JARs are found late**: `NoClassDefFoundError` at run time, maybe after hours.
- **Duplicates**: two versions of a class, the first one wins silently.
- **The whole JDK** is always there: about 70 modules, even for "Hello".

## Module (Java 9, 2017)

A named group of packages with a `module-info.java`:

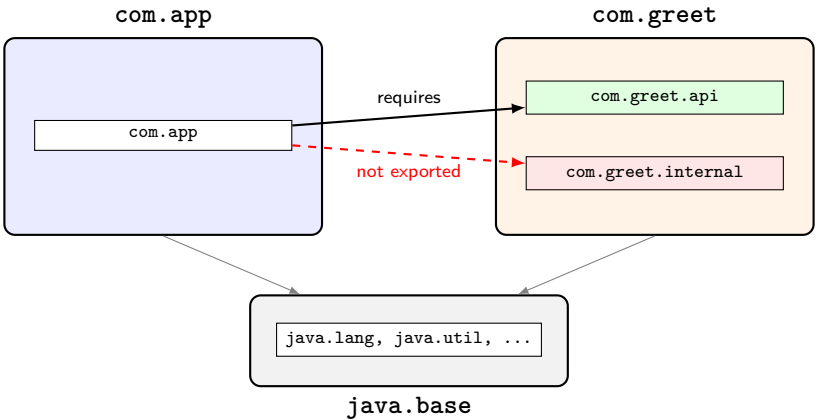
- `requires`: what it needs
- `exports`: what others may use

Checked by `javac` **and** by `java` at start-up.

## The JDK itself

`java.base`, `java.sql`, `java.desktop`, ...
Every module reads `java.base`.

# Two modules




Green: exported, usable by `com.app`. Red: hidden even though `Words` is `public`. Grey: `requires java.base` is implicit.

# The source tree: one folder per module

codeoutput/Project/src/com.greet/module-info.java Java

```
1 module com.greet {
2     exports com.greet.api;          // public to other modules
3 }                                   // com.greet.internal stays hidden
```

codeoutput/Project/src/com.app/module-info.java Java

```
1 module com.app {
2     requires com.greet;
3 }
```

codeoutput/Project/src/com.app/com/app/Main.java Java

```
1 package com.app;
2
3 import com.greet.api.Greeter;
4
5 public class Main {
6     public static void main(String[] args) {
7         System.out.println(Greeter.hello("Yeditepe"));
8     }
9 }
```

- `src/com.greet` and `src/com.app`: one folder per module, named after the module.
- Inside it, `module-info.java` at the top and the packages as folders, as before.
- Module names look like package names (reverse domain); the two need not be equal.

# The library classes

codeoutput/Project/src/com.greet/com/greet/internal/Words.java — Java

```java
package com.greet.internal;

public class Words {
    public static String hello() {
        return "Merhaba";
    }
}
```

codeoutput/Project/src/com.greet/com/greet/api/Greeter.java — Java

```java
package com.greet.api;

import com.greet.internal.Words;

public class Greeter {
    public static String hello(String name) {
        return Words.hello() + ", " + name + "!";
    }
}
```

# Compiling and running modules

**Terminal** codeoutput/Modules.txt

```
$ find src -name "*.java" | sort
src/com.app/com/app/Main.java
src/com.app/module-info.java
src/com.greet/com/greet/api/Greeter.java
src/com.greet/com/greet/internal/Words.java
src/com.greet/module-info.java
$ javac -d mods --module-source-path src -m com.greet,com.app
$ find mods -type f | sort
mods/com.app/com/app/Main.class
mods/com.app/module-info.class
mods/com.greet/com/greet/api/Greeter.class
mods/com.greet/com/greet/internal/Words.class
mods/com.greet/module-info.class
$ java --module-path mods --module com.app/com.app.Main
Merhaba, Yeditepe!
$ java -p mods -m com.app/com.app.Main
Merhaba, Yeditepe!
$ java -p mods --describe-module com.greet
com.greet file:///~/project/mods/com.greet/
exports com.greet.api
requires java.base mandated
contains com.greet.internal
```

`-module-source-path src` finds every module under `src`; `-m` names the modules to compile. Run with `-p` (`-module-path`) and `-m module/main.class`. **Windows**: same commands; `-p` lists use `;`.

# What `java -p mods -m com.app` does first

**Algorithm 1:** Resolving modules at start-up

**Input:** root module $r$, module path $p$

**Output:** the set $R$ of modules to load, or an error **before** `main` runs

1 $R \leftarrow \emptyset$;
2 $Q \leftarrow \langle r, \texttt{java.base} \rangle$;
3 **while** $Q$ *is not empty* **do**
4 $\quad m \leftarrow \textsc{Dequeue}(Q)$;
5 $\quad$ **if** $m \in R$ **then continue**;
6 $\quad$ **if** $m$ *is neither in the JDK nor on* $p$ **then fail** `FindException: Module m not found`;
7 $\quad R \leftarrow R \cup \{m\}$;
8 $\quad$ **foreach** *requires* $n$ *of* $m$ **do**
9 $\quad\quad \textsc{Enqueue}(Q, n)$;
10 check: no package in two modules of $R$, no cycles;
11 **return** $R$;

# Forgetting `requires`

**Terminal** codeoutput/NoRequires.txt

```
$ cat src/com.app/module-info.java
module com.app {
}
$ javac -d mods --module-source-path src -m com.greet,com.app 2>&1 | head -4
src/com.app/com/app/Main.java:3: error: package com.greet.api is not visible
import com.greet.api.Greeter;
                  ^
  (package com.greet.api is declared in module com.greet, but module com.app does not read it)
```

Without `requires com.greet`, module `com.app` does not *read* `com.greet`: nothing of it is visible, even exported packages.

# Using a hidden package

codeoutput/Illegal/Main.java **Java**

```java
package com.app;

import com.greet.internal.Words;   // not
        exported!

public class Main {
    public static void main(String[] args) {
        System.out.println(Words.hello());
    }
}
```

`public` is not enough: the package must be **exported**. `-add-exports` (and `-add-opens` for reflection) break the rule on purpose, at compile *and* at run time: a last resort for old code.

# Using a hidden package

**codeoutput/Illegal/Main.java Java**

```java
package com.app;

import com.greet.internal.Words;   // not exported!

public class Main {
    public static void main(String[] args) {
        System.out.println(Words.hello());
    }
}
```

**Terminal** codeoutput/Encapsulation.txt

```
$ javac -d mods --module-source-path src -m com.greet,com.app
src/com.app/com/app/Main.java:3: error: package com.greet.internal is not visible
import com.greet.internal.Words;   // not exported!
                         ^
  (package com.greet.internal is declared in module com.greet, which does not export it
      )
1 error
$ javac -d mods --module-source-path src -m com.greet,com.app --add-exports com.greet/
      com.greet.internal=com.app
$ java -p mods -m com.app/com.app.Main
Exception in thread "main" java.lang.IllegalAccessError: class com.app.Main (in module
      com.app) cannot access class com.greet.internal.Words (in module com.greet)
      because module com.greet does not export com.greet.internal to module com.app
       at com.app/com.app.Main.main(Main.java:7)
$ java -p mods --add-exports com.greet/com.greet.internal=com.app -m com.app/com.app.
      Main
Merhaba
```

`public` is not enough: the package must be **exported**. `-add-exports` (and `-add-opens` for reflection) break the rule on purpose, at compile *and* at run time: a last resort for old code.

# Modular JARs

# A modular JAR = a JAR with `module-info.class`

**Terminal** codeoutput/ModJar.txt

```
$ mkdir mlib
$ jar --create --file mlib/greet.jar -C mods/com.greet .
$ jar --create --file mlib/app.jar --main-class com.app.Main -C mods/com.app .
$ jar --describe-module --file mlib/app.jar
com.app jar:file://~/project/mlib/app.jar!/module-info.class
requires com.greet
requires java.base mandated
contains com.app
main-class com.app.Main

$ java -p mlib -m com.app
Merhaba, Yeditepe!
$ java -cp "mlib/*" com.app.Main
Merhaba, Yeditepe!
$ jdeps --module-path mlib -summary mlib/app.jar
com.app -> com.greet
com.app -> java.base
```

The same JAR works on the **module path** (`-p`, checked) and on the **class path** (`-cp`, `module-info` ignored). A plain JAR on `-p` becomes an *automatic module* named after the file.

# Class path or module path?

| | class path `-cp` | module path `-p` |
|---|---|---|
| unit | folders, JARs | modules (modular or automatic JARs) |
| missing dependency | `NoClassDefFoundError`, late | `FindException`, at start-up |
| two copies | first one wins | error |
| hidden packages | none | everything not `exportsed` |
| run | `java -cp a.jar p.Main` | `java -p mlib -m mod/p.Main` |
| custom runtime | no | `jlink` |

Most libraries are still used on the class path; new applications benefit from modules mostly when they want `jlink`.

# Custom Runtimes: jlink

# A runtime with only the modules we need

**Terminal** codeoutput/Jlink.txt

```
$ java --list-modules | wc -l
69
$ jdeps --module-path mlib --print-module-deps mlib/app.jar
com.greet,java.base
$ jlink --module-path mlib --add-modules com.app --launcher hello=com.app --strip-debug --no-header-files --no-man-pages --output image
$ ls image/bin
hello
java
keytool
$ image/bin/java --list-modules
com.app
com.greet
java.base@21.0.12.1
$ image/bin/hello
Merhaba, Yeditepe!
$ du -sh image
52M     image
$ jlink --add-modules java.se --output full
$ du -sh full
101M    full
```

`jdeps -print-module-deps` tells which JDK modules are needed. The image has its own `bin/java` and a `hello` launcher; it does **not** need an installed JDK. Compare its size with an image of all of Java SE (`-compress zip-6` makes both smaller).

# jlink on Windows and macOS

## Windows (Command Prompt)

```
jlink --module-path mlib ^
      --add-modules com.app ^
      --launcher hello=com.app ^
      --strip-debug --no-header-files ^
      --no-man-pages --output image
image\bin\hello
```

## macOS (Terminal)

```
jlink --module-path mlib \
      --add-modules com.app \
      --launcher hello=com.app \
      --strip-debug --no-header-files \
      --no-man-pages --output image
image/bin/hello
```

- The image is made for **one** operating system and CPU: build it on Windows for Windows, on macOS for macOS (Apple silicon `aarch64` and Intel `x64` differ).
- `jlink` needs the JDK's `jmods`; it only accepts modules, so every JAR must be modular.

# Installers: jpackage

# An application folder with a native launcher

**Terminal** codeoutput/Jpackage.txt

```
$ jpackage --type app-image --name Hello --module-path mlib --module com.app --dest out
$ find out -maxdepth 3 | sort
out
out/Hello
out/Hello/bin
out/Hello/bin/Hello
out/Hello/lib
out/Hello/lib/Hello.png
out/Hello/lib/app
out/Hello/lib/libapplauncher.so
out/Hello/lib/runtime
$ out/Hello/bin/Hello
Merhaba, Yeditepe!
$ du -sh out/Hello
53M     out/Hello
```

`jpackage` runs `jlink` for us, adds a native launcher (`bin/Hello`) and puts the JARs into `lib/app`. `-type app-image` works everywhere; the other types make a real installer.

# Installers for Windows and macOS

## Windows (Command Prompt)

```
jpackage --type msi --name Hello ^
  --module-path mlib --module com.app ^
  --win-menu --win-shortcut
rem -> Hello-1.0.msi  (also: --type exe)
```

## macOS (Terminal)

```
jpackage --type dmg --name Hello \
  --module-path mlib --module com.app
# -> Hello-1.0.dmg  (also: --type pkg)
# signing: --mac-sign ...
```

- **Windows**: `msi`/`exe` need the free **WiX Toolset** on the `PATH`. Installs into `C:\Program Files\Hello`.
- **macOS**: the `.dmg` contains `Hello.app`. Unsigned apps are blocked by Gatekeeper; for others to run it, sign and notarize it with an Apple Developer ID.
- Linux: `-type deb` or `rpm`. There is **no cross-packaging**: run `jpackage` on each target system (often in CI).

# Which way to ship?

| deliver | user needs | size | made with |
|---|---|---|---|
| JAR (+ `lib/`) | a Java runtime | KB | `jar` |
| fat JAR | a Java runtime | KB–MB | `jar`, Maven Shade |
| runtime image | nothing | 30–60 MB | `jlink` |
| app image / installer | nothing | 40–70 MB | `jpackage` |
| container image | Docker / Podman | 100+ MB | `Dockerfile` (senior deck) |

Libraries are shipped as JARs (to Maven Central); desktop applications as installers; services as containers.

# Multi-Release JARs

# One JAR, different code for different Java versions

**Terminal** codeoutput/MRJar.txt

```
$ find . -name "*.java" | sort
./src-17/Info.java
./src/Info.java
$ javac --release 8 -Xlint:-options -d classes src/Info.java
$ javac --release 17 -d classes-17 src-17/Info.java
$ jar --create --file info.jar --main-class Info -C classes . --release 17 -C classes-17 .
$ jar tf info.jar
META-INF/
META-INF/MANIFEST.MF
Info.class
META-INF/versions/17/
META-INF/versions/17/Info$1Point.class
META-INF/versions/17/Info.class
$ unzip -p info.jar META-INF/MANIFEST.MF | grep Multi
Multi-Release: true
$ java -jar info.jar
Java 17 version: Point[x=3, y=4]
```

Classes under `META-INF/versions/17/` replace the root ones on Java 17 and newer; Java 8–16 use the root classes. `jar -release 17` puts them there and writes `Multi-Release: true` into the manifest.

# Tuning the JVM from the Command Line

# Kinds of options

| kind | example | note |
|---|---|---|
| standard | `-cp`, `-p`, `-D`, `-ea`, `-version` | stable, `java -help` |
| extra `-X` | `-Xmx512m`, `-Xss1m`, `-Xlog:gc` | `java -X` |
| advanced `-XX:` | `-XX:+UseZGC`, `-XX:+HeapDumpOnOutOfMemoryError` | may change between versions |
| environment | `JDK_JAVA_OPTIONS` | added to every `java` command |

- `-Xms`/`-Xmx`: initial/maximum heap. Default maximum: $\frac{1}{4}$ of the physical memory.
- Options go **before** the class or `-jar` file; everything after it is an argument of `main`.
- **Windows**: `set JDK_JAVA_OPTIONS=-Xmx512m`; **macOS**: `export JDK_JAVA_OPTIONS=-Xmx512m`.

# Heap size in practice

codeoutput/JvmOptions/Hog.java **Java**

```java
import java.util.ArrayList;
import java.util.List;

public class Hog {
    public static void main(String[] args) {
        List<long[]> keep = new ArrayList<>();
        while (true) {
            keep.add(new long[1_000_000]);    // 8 MB each
            System.out.println("blocks: " + keep.size());
        }
    }
}
```

`JDK_JAVA_OPTIONS` is announced with `NOTE: Picked up ...` so you know it was used. With a 32 MB heap the program fails quickly with `OutOfMemoryError`.

# Heap size in practice

codeoutput/JvmOptions/Hog.java — Java

```java
import java.util.ArrayList;
import java.util.List;

public class Hog {
    public static void main(String[] args) {
        List<long[]> keep = new ArrayList<>();
        while (true) {
            keep.add(new long[1_000_000]);    // 8 MB each
            System.out.println("blocks: " + keep.size());
        }
    }
}
```

Terminal — codeoutput/JvmOptions.txt

```
$ java -XshowSettings:vm -version 2>&1 | head -2
VM settings:
    Max. Heap Size (Estimated): 1.46G
$ java -Xmx64m -XshowSettings:vm -version 2>&1 | head -2
VM settings:
    Max. Heap Size: 64.00M
$ export JDK_JAVA_OPTIONS=-Xmx32m
$ java -XshowSettings:vm -version 2>&1 | head -3
NOTE: Picked up JDK_JAVA_OPTIONS: -Xmx32m
VM settings:
    Max. Heap Size: 32.00M
$ unset JDK_JAVA_OPTIONS
$ java -XX:+PrintFlagsFinal -version 2>/dev/null | grep -E " (MaxHeapSize|UseG1GC)
       " | awk '{print $2, $4}'
MaxHeapSize 1568669696
UseG1GC true
$ javac Hog.java
$ java -Xmx32m Hog
blocks: 1
blocks: 2
blocks: 3
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
        at Hog.main(Hog.java:8)
```

`JDK_JAVA_OPTIONS` is announced with `NOTE: Picked up ...` so you know it was used. With a 32 MB heap the program fails quickly with `OutOfMemoryError`.

# jps and jcmd

codeoutput/Monitor/Server.java **Java**

```
1 public class Server {
2     public static void main(String[] args) throws
          InterruptedException {
3         Thread.sleep(60_000);                    //
          pretend to serve for a minute
4     }
5 }
```

`jps` lists running JVMs with their process id; `jcmd <pid or main class> <command>` asks one of them for information (`jcmd Server help` lists the commands). The same tools work on Windows and macOS.

# jps and jcmd

codeoutput/Monitor/Server.java — Java

```java
public class Server {
    public static void main(String[] args) throws
        InterruptedException {
        Thread.sleep(60_000);                    //
        pretend to serve for a minute
    }
}
```

Terminal — codeoutput/Monitor.txt

```
$ java -Xmx64m Server &
$ sleep 1
$ jps
6195 Jps
6174 Server
$ jcmd Server VM.version
6174:
OpenJDK 64-Bit Server VM version 21.0.12.1+1-1-24.04.4-Ubuntu
JDK 21.0.12.1
$ jcmd Server VM.flags | tr " " "\n" | grep MaxHeap
-XX:MaxHeapSize=67108864
-XX:SoftMaxHeapSize=67108864
$ jcmd Server Thread.print | grep -c "java.lang.Thread.State"
11
$ jcmd Server GC.heap_info | head -3 | cut -c1-70
6174:
 garbage-first heap   total 65536K, used 1688K [0x00000000fc000000, 0x
  region size 1024K, 1 young (1024K), 0 survivors (0K)
```

`jps` lists running JVMs with their process id; `jcmd <pid or main class> <command>` asks one of them for information (`jcmd Server help` lists the commands). The same tools work on Windows and macOS.

# Build Tools and Modules

# Maven and Gradle with modules

## Maven: `mvn package`

```
src/main/java/module-info.java
src/main/java/com/app/Main.java
# the compiler plugin sees module-info and
# puts the dependencies on the module path
mvn package
java -p target/app.jar:libs -m com.app
```

## Gradle: `build.gradle.kts`

```
plugins { application }
application {
    mainModule = "com.app"
    mainClass  = "com.app.Main"
}
// ./gradlew run   or   ./gradlew installDist
```

Not run here (they need the internet). Plugins such as `org.beryx.jlink` or `maven-jlink-plugin` call `jlink` and `jpackage` for you; the options are the ones on these slides.

# Summary

# Recipes

| goal | command (**Windows**: ; in paths, ^ to continue a line) |
|---|---|
| compile modules | `javac -d mods -module-source-path src -m a,b` |
| run a module | `java -p mods -m b/b.Main` |
| describe a module | `java -p mods -describe-module a` |
| modular JAR | `jar -create -file mlib/b.jar -main-class b.Main -C mods/b .` |
| needed JDK modules | `jdeps -print-module-deps -module-path mlib mlib/b.jar` |
| custom runtime | `jlink -p mlib -add-modules b -launcher b=b -output image` |
| installer | `jpackage -type msi\|dmg\|deb -name B -p mlib -m b` |
| multi-release JAR | `jar -create -file x.jar -C classes .  -release 17 -C c17 .` |
| heap and options | `java -Xmx512m -XshowSettings:vm ...`, `JDK_JAVA_OPTIONS` |
| running JVMs | `jps`, `jcmd <pid> VM.flags` |

# Exercises

1. Turn the sophomore project (`util` + `books`) into two modules. Which packages must be exported?
2. Remove `exports` from `com.greet` and read the error. Then fix it with `--add-exports` only on the command line.
3. Make a `jlink` image of the books application. How big is it? Does it start faster than `java -jar`? (`time`)
4. Build an installer with `jpackage` on your own laptop (`msi` on Windows, `dmg` on macOS) and install it.
5. Write a multi-release JAR whose Java 21 version uses a `switch` with patterns.
6. Run `Hog` with `-Xmx16m`, `-Xmx64m`, `-Xmx256m`; plot the number of blocks against the heap.

# Questions?